



Sistemas Operacionais — Atividade 1

Processos, Threads, Escalonamento e Inferência Local com Ollama

Trilha A — Chat local: Ollama + Open WebUI

· Modelo: Granite-3.1-3B-A800M-Instruct (MoE)

Equipe: GT 08

Integrantes: Mikael Douglas Santos Farias, Cadmo Otávio José do Nascimento Neto, Mateus do Rosario Costa, Douglas de Oliveira Déda, Paulo Gabriel de Oliveira Cardoso, Lucas Conceicao Celestino.

Professor(a): GLAUCO DE FIGUEIREDO CARNEIRO

Disciplina: Sistemas Operacionais — AV1 — 2026.2

Data: 15/09/2026

1. Contribuição de cada integrante

A tabela abaixo indica as atividades realizadas por cada integrante da equipe, conforme exigido pela atividade.

Integrante	Contribuições
Lucas Conceição Celestino	Ajuda no setup do WSL2/Docker, Documentação, Slide, Relatórios, Escolha do modelo
Mateus do Rosario Costa	WSL2/Docker, scripts de benchmark e geração de gráficos
Cadmo Octávio José do Nascimento Neto	Análise de resultados obtidos das médias entre GPU e CPU
Mikael Douglas Santos Farias	Coordenação da atividade; organização do relatório técnico; definição do contexto, objetivo, Trilha A, ambiente experimental e justificativa do modelo;

2. Introdução e problema

Aplicações de IA generativa executadas localmente não envolvem apenas um modelo de linguagem: elas mobilizam processos, threads, memória, armazenamento, comunicação em rede, arquivos, logs, bibliotecas, runtimes e mecanismos de proteção do sistema operacional. Esta atividade tem como objetivo relacionar esses conceitos de Sistemas Operacionais a uma aplicação real de IA generativa executada em ambiente Linux (via WSL2), utilizando o Ollama como runtime de inferência e o Open WebUI como camada de aplicação (Trilha A).

Pergunta norteadora: como a camada de aplicação, o modelo de linguagem, a quantidade de parâmetros, a quantização e a configuração de execução (com e sem GPU) afetam processos, threads, uso de CPU, memória e responsividade de um sistema local de IA generativa?

3. Trilha e camada de aplicação selecionadas

Trilha selecionada: Trilha A — Chat local: Ollama + Open WebUI.

O Open WebUI é uma plataforma auto-hospedada e extensível, capaz de operar offline, com integração nativa a runtimes locais como o Ollama. Essa trilha permite estudar a interação entre uma interface web, serviços locais, comunicação via API HTTP, contêineres, portas, volumes persistentes e execução de modelos — cobrindo diretamente os objetivos de aprendizagem da atividade.

Justificativa da escolha da trilha:

A equipe optou pela Trilha A (Ollama + Open WebUI) por ela oferecer o cenário mais representativo de um sistema de IA generativa em uso real: uma interface web servindo múltiplos usuários simultaneamente, comunicando-se com o runtime de inferência via API HTTP. Diferente das trilhas B e C, cujo foco está concentrado nas etapas de indexação e recuperação de documentos (RAG), a Trilha A permite observar todo o ciclo de vida de requisições concorrentes — desde a abertura de conexões e sockets até o escalonamento de processos e threads sob carga — o que se conecta diretamente aos tópicos centrais da disciplina de Sistemas Operacionais, como concorrência, gerência de processos e comunicação entre componentes.

4. Modelo Hugging Face selecionado

Item	Valor
Nome completo do modelo	ibm-granite/granite-3.1-3b-a800m-instruct
URL do model card	https://huggingface.co/ibm-granite/granite-3.1-3b-a800m-instruct

Arquitetura	Mixture of Experts (MoE)
Número de parâmetros totais	3 bilhões (3B)
Parâmetros ativos por token	~800 milhões (A800M)
Variante	Instruct
Formato	GGUF
Quantização utilizada	Q4 K M (~2,0 GB, conforme `ollama list`)
Licença	Apache 2.0
Nome no Ollama	granite3.1-moe:3b
Contexto máximo	128.000 tokens

Observação metodológica: a equipe iniciou os experimentos com o modelo Phi-4-mini-instruct (denso, 3,8B) e trocou para o Granite-3.1-3B-A800M-Instruct (MoE) com autorização do professor, reaproveitando toda a infraestrutura (WSL2, Docker, NVIDIA Container Toolkit, containers) já validada. Os números deste relatório referem-se exclusivamente ao Granite; a comparação com os resultados do Phi-4-mini é retomada na Seção 11 e na Conclusão, onde é relevante.

Justificativa curta da escolha:

O Granite-3.1-3B-A800M-Instruct foi escolhido por ser uma arquitetura Mixture of Experts (MoE): embora possua 3B de parâmetros totais, ativa apenas ~800M por token, favorecendo respostas rápidas com uso eficiente de CPU/GPU. Sua licença Apache 2.0 é totalmente permissiva, e o contexto de 128K tokens é equivalente ao de modelos maiores. A escolha também permitiu comparar, na prática, o comportamento de um modelo MoE com o de um modelo denso já testado anteriormente (Phi-4-mini), enriquecendo a análise de processos, threads e uso de recursos pedida pela atividade.

5. URL do projeto público criado no GitHub

https://github.com/LucasCelestinoSE/SO_UFS_2026

6. Ambiente experimental

Componente	Detalhe
Sistema operacional host	Windows com WSL2 (Ubuntu 26.04 LTS "Resolute Raccoon")
Kernel	6.18.33.2-microsoft-standard-WSL2 (x86_64)
Processador	Intel(R) Xeon(R) CPU E5-2680 v4 @ 2.40GHz — 28 CPUs lógicas (14 núcleos × 2 threads)
Container runtime	Docker Engine 29.8.0 (instalação nativa dentro do WSL2)
GPU	NVIDIA GeForce RTX 3060 — 12.288 MiB (12 GB) de VRAM
Driver / CUDA	Driver NVIDIA 615.65.07 · CUDA UMD 13.4 (via passthrough WSL2)
Suporte a GPU no Docker	NVIDIA Container Toolkit (nvidia-ctk)
RAM total (WSL2)	15 GiB (conforme `free -h`)
Armazenamento	1 TB em /var/lib/docker (disco virtual dedicado do WSL2), 924 GB livres no momento da coleta
Runtime de inferência	Ollama 0.34.0 (imagem oficial ollama/ollama), dois containers independentes
Camada de aplicação	Open WebUI (ghcr.io/open-webui/open-webui:main), rede host, porta 8080
Linguagem/dependências	Python 3.14.4 (scripts de benchmark e gráficos)

Modelo	granite3.1-moe:3b (Granite-3.1-3B-A800M-Instruct), ~2,0 GB em disco
--------	--

O inventário completo do ambiente foi coletado de forma automatizada com o script ``coletar_inventario.sh`` (Seção 6.1 da atividade), que executa ``uname``, ``lscpu``, ``free``, ``df``, ``nvidia-smi``, além de consultar a versão e a lista de modelos do Ollama dentro de cada container. O arquivo consolidado (``inventario.md``) está disponível na pasta ``inventario/`` do repositório.

Para permitir a comparação controlada entre execução com e sem GPU (Configuração 3, Seção 9), foram reaproveitados os dois containers Ollama independentes já criados no experimento anterior com o modelo Phi-4-mini, com o mesmo volume de dados isolado e a mesma versão de imagem: `ollama-gpu` (porta 11434, com a flag `--gpu=all`) e `ollama-cpu` (porta 11435, sem acesso à GPU). A infraestrutura foi posteriormente declarada em um arquivo `docker-compose.yml`, permitindo reconstruir os três serviços (`ollama-gpu`, `ollama-cpu`, `open-webui`) de forma reprodutível a partir dos mesmos volumes.

7. Arquitetura da aplicação

A arquitetura é composta por três contêineres Docker independentes, todos na rede host do WSL2:

- `ollama-gpu` — servidor Ollama com acesso à GPU via NVIDIA Container Toolkit, escutando na porta 11434, com volume persistente `ollama-gpu-data`.
- `ollama-cpu` — servidor Ollama idêntico, porém sem acesso à GPU, escutando na porta 11435, com volume persistente `ollama-cpu-data`.
- `open-webui` — interface web de chat, escutando na porta 8080, comunicando-se via HTTP com um dos dois servidores Ollama através da variável `OLLAMA_BASE_URL`.

Cada servidor Ollama carrega o modelo `granite3.1-moe:3b` sob demanda e o mantém residente em memória (RAM ou VRAM) entre requisições consecutivas. A comunicação entre a camada de aplicação e o runtime ocorre inteiramente via API HTTP local (`POST /api/generate`), sem dependência de serviços externos.

8. Processos, threads e chamadas de sistema

8.1 Processos e threads observados

A inspeção com ``docker exec <container> ps -eLf`` revelou, em ambos os containers Ollama (GPU e CPU), um único processo principal — ``/bin/ollama serve``, executando como PID 1 dentro do namespace do container — responsável por todo o ciclo de vida do servidor. Esse processo mantém um pool de threads (36 no container GPU, 35 no container CPU — NLWP), evidenciando o mesmo modelo de concorrência baseado em threads dentro de um único processo observado no experimento anterior com o Phi-4-mini (que usava 37 threads em ambos), sem hierarquia de processos filhos.

Container	PID (namespace)	Nº de threads (NLWP)	Comando
ollama-gpu	1	36	<code>/bin/ollama serve</code>
ollama-cpu	1	35	<code>/bin/ollama serve</code>

Já o container `open-webui` apresentou 130 PIDs (conforme ``docker stats``), consistente com sua pilha multi-processo (backend Python/FastAPI e demais serviços da interface), contrastando com o modelo single-process/multi-thread do Ollama.

8.2 Chamadas de sistema (strace)

O rastreamento com ``strace -f -c`` foi feito do lado do host, anexado ao PID do processo do Ollama dentro do container `ollama-gpu` (``strace -f -c -p PID``), durante uma inferência via Open WebUI. O processo foi encontrado com 11 threads no momento da anexação (uma janela mais curta que a observada via ``ps -eLf`` no namespace do container, de 36 threads —

a diferença reflete quantas threads estavam de fato ativas no instante em que o strace anexou, não a capacidade total do pool). O resumo (tempo total de amostragem: $\approx 0,31$ s, bem mais curto que os 200s capturados no experimento anterior com o Phi-4-mini) foi:

Chamada	% tempo	Nº de chamadas	Relação com a execução
futex	74,16%	118	Sincronização entre as threads do pool (contenção de locks sob carga)
nanosleep	10,11%	187	Threads ociosas aguardando trabalho em intervalos curtos (polling do agendador interno)
epoll_pwait	7,86%	112	Multiplexação de E/S assíncrona do servidor HTTP
read	3,09%	103	Leitura de dados da requisição/conexão TCP
write	2,89%	55	Escrita da resposta gerada na conexão TCP

Assim como no experimento com o Phi-4-mini, `futex` continua sendo a chamada dominante (74,16% do tempo), confirmando que a sincronização entre threads é o custo central também no modelo MoE. A proporção é um pouco menor que os 84,68% observados com o Phi-4-mini, e chamadas como `accept4` e `mmap` não aparecem nesta amostra — provavelmente porque a captura foi mais curta e não coincidiu com o momento de aceitação de uma nova conexão ou de carregamento inicial do modelo (que já estava residente em memória). Por essa diferença na janela de captura, a comparação direta em número absoluto de chamadas entre os dois experimentos deve ser vista com cautela; a comparação percentual de `futex` é a mais robusta entre as duas amostras.

8.3 Portas e comunicação

Porta	Serviço
8080	Open WebUI (interface de chat)
11434	Ollama — container ollama-gpu
11435	Ollama — container ollama-cpu

9. Configurações e metodologia

Seguindo a Parte C da atividade (Experimentos comparativos), foram executadas três configurações, totalizando 18 execuções mensuráveis — acima do mínimo de 12 exigido:

Configuração 1 — Execução padrão

5 perguntas (mistura de curtas e longas) enviadas sequencialmente, uma de cada vez, via API do Ollama (POST /api/generate, stream=false), repetidas nos containers GPU e CPU (10 execuções).

Configuração 2 — Concorrência/carga controlada

As mesmas perguntas disparadas simultaneamente via ThreadPoolExecutor em 4 níveis de concorrência (2, 4, 6 e 12 requisições simultâneas), em cada container (8 rodadas).

Configuração 3 — Ajuste de execução local: CPU versus GPU

Todas as execuções acima foram duplicadas entre o container ollama-gpu (--gpus=all) e o container ollama-cpu (sem acesso à GPU), mantendo o mesmo modelo-base (granite3.1-moe:3b), o mesmo formato e a mesma quantização — variando apenas a disponibilidade de GPU, conforme permitido pela atividade.

Ferramentas de medição utilizadas: script Python (benchmark_gpu_vs_cpu.py) com requests + ThreadPoolExecutor para disparo e cronometragem das requisições, e docker stats para CPU%/memória por container; nvidia-smi para utilização e VRAM da GPU; ps -eLf e strace para processos, threads e chamadas de sistema.

10. Resultados

10.1 Execução sequencial (uma requisição por vez)

Config.	Duração média (s)	Tokens/s médio
GPU	3,41	167,74
CPU	13,00	42,03

Com GPU, o tempo médio de resposta foi $3,81\times$ mais rápido, e a geração de tokens foi $3,99\times$ mais rápida que apenas em CPU.

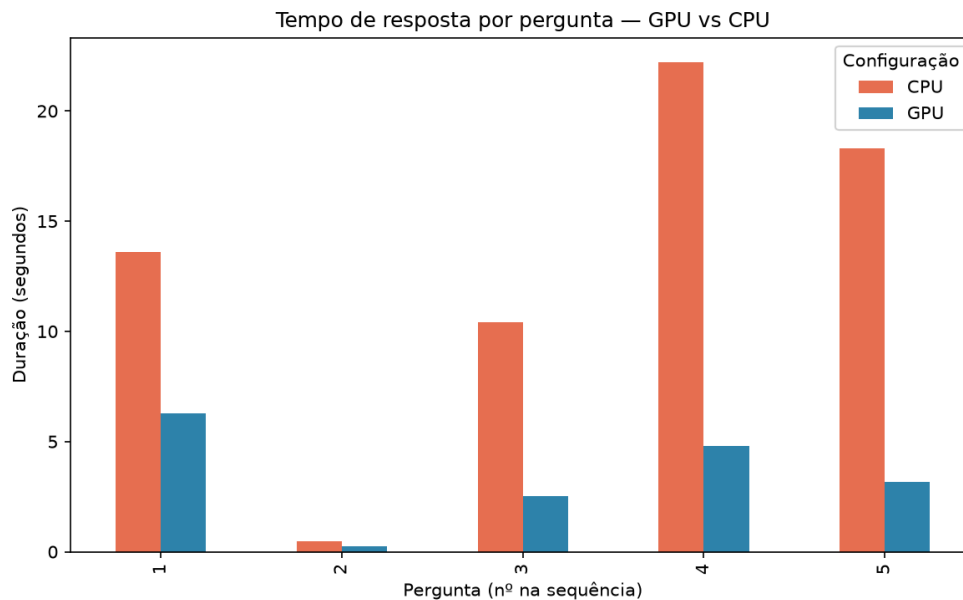


Figura 1 — Tempo de resposta por pergunta, GPU vs. CPU.

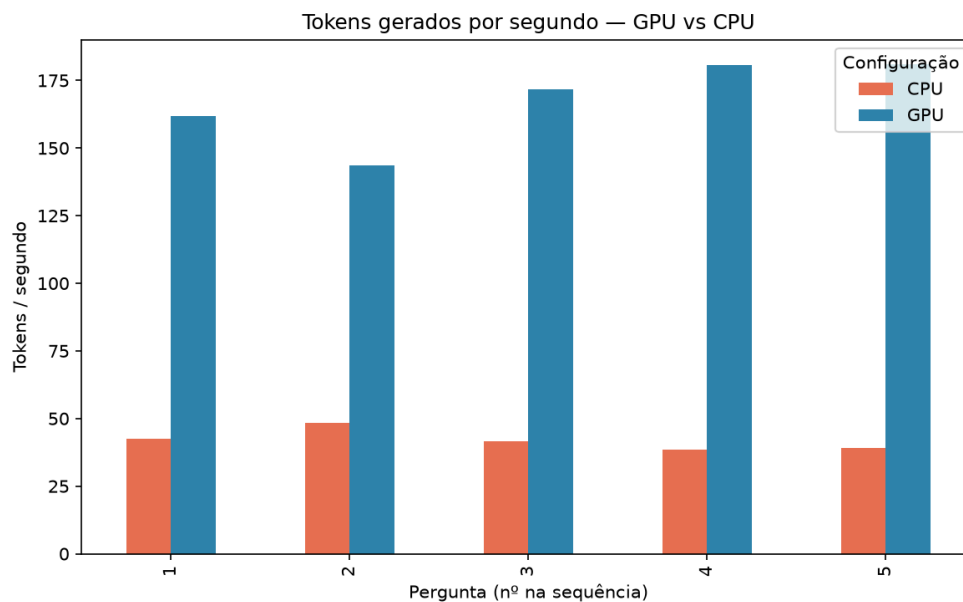


Figura 2 — Tokens gerados por segundo, GPU vs. CPU.

10.2 Concorrência (2, 4, 6 e 12 requisições simultâneas)

Nível	Duração média/req GPU (s)	Duração média/req CPU (s)	Throughput GPU (req/s)	Throughput CPU (req/s)
2	1,10	3,74	1,667	0,503

4	4,88	22,08	0,409	0,085
6	8,13	34,52	0,374	0,093
12	13,35	63,21	0,470	0,111

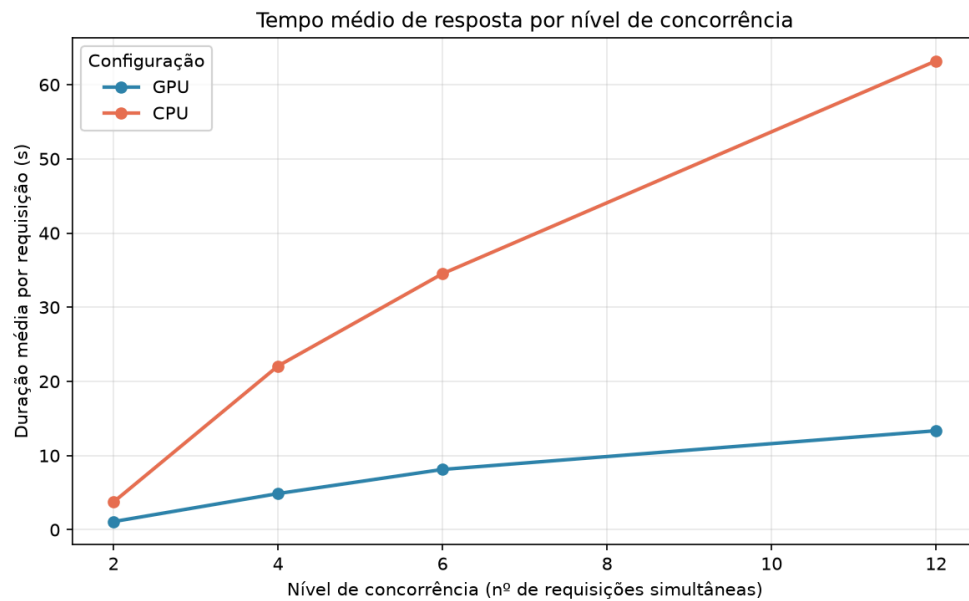


Figura 3 — Tempo médio de resposta por nível de concorrência.

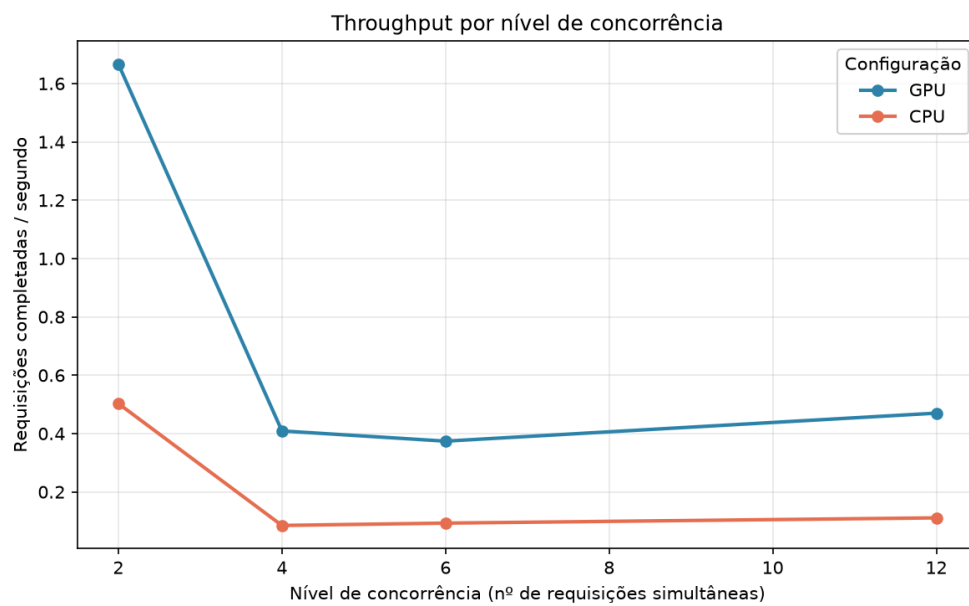


Figura 4 — Throughput (requisições/segundo) por nível de concorrência.

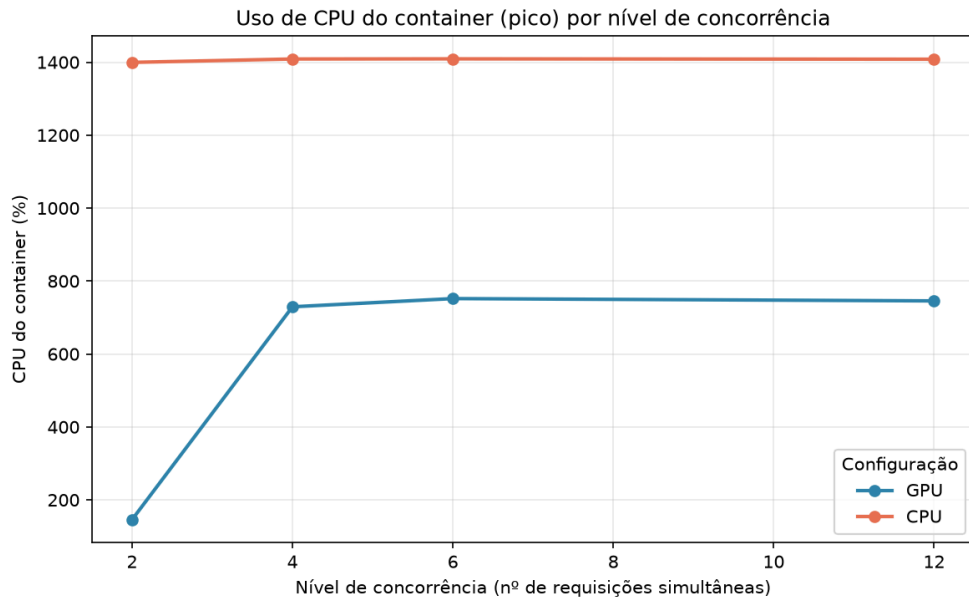


Figura 5 — *Uso de CPU do container (pico) por nível de concorrência.*

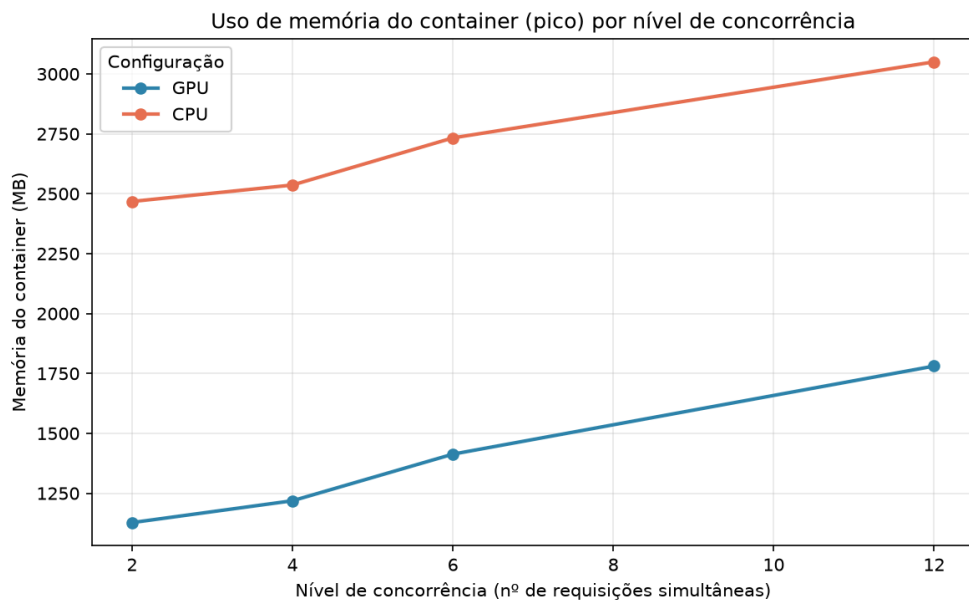


Figura 6 — *Uso de memória do container (pico) por nível de concorrência.*

10.3 GPU: utilização e VRAM durante a inferência

Durante uma inferência no container ollama-gpu, o `nvidia-smi` registrou 60% de utilização da GPU e 3.375 MiB de VRAM ocupados (de 12.288 MiB disponíveis, $\approx 27\%$), com o processo `/llama-server` — o processo interno do Ollama responsável pela execução do modelo — listado como consumidor da GPU. Tanto a utilização quanto a VRAM ocupada são menores que as observadas com o Phi-4-mini (87% e $\approx 33\%$), consistente com o Granite ativar apenas $\sim 800\text{M}$ de parâmetros por token.

11. Discussão

Esta seção responde às questões de análise exigidas pela atividade (Seção 9 do documento), com base nas evidências coletadas.

1. Qual trilha foi selecionada e por que é apropriada?

Trilha A (Ollama + Open WebUI). É apropriada porque expõe diretamente conceitos de SO como processos multi-thread, comunicação via API HTTP, portas, contêineres e uso de GPU/CPU, todos observáveis com ferramentas padrão do Linux (ps, ss, docker stats, nvidia-smi, strace).

2. Qual modelo foi selecionado, parâmetros e por quê?

Granite-3.1-3B-A800M-Instruct (IBM), arquitetura Mixture of Experts: 3B de parâmetros totais, ~800M ativos por token. Trocado a partir do Phi-4-mini (denso, 3,8B) com autorização do professor, para explorar na prática o comportamento de um modelo MoE.

3. Como formato, quantização, contexto e tamanho afetaram RAM/VRAM?

O formato GGUF com quantização Q4_K_M reduz o modelo a ~2,0 GB em disco. Em execução, o container GPU usou entre ~1,1 GB e ~1,8 GB de RAM (crescendo com a concorrência) e ~3,4 GB de VRAM (~27% dos 12 GB da RTX 3060); o container CPU usou entre ~2,5 GB e ~3,1 GB de RAM, sem uso de VRAM. Os valores de VRAM e utilização de GPU ficaram abaixo dos observados com o Phi-4-mini (4,1 GB/87%), coerente com o menor número de parâmetros ativos por token do MoE.

4/5. Quais processos/threads foram observados e sua relação de parentesco?

Um único processo (`ollama serve`, PID 1 no namespace do container) com pool de threads — 36 no container GPU, 35 no CPU — modelo single-process/multi-thread, sem hierarquia de processos filhos, no mesmo padrão observado com o Phi-4-mini (que usava 37 threads em ambos).

6. Quais chamadas de sistema foram relevantes?

``futex`` (74,16% do tempo) para sincronização entre threads — o mesmo padrão dominante do experimento anterior; ``nanosleep`` e ``epoll_pwait`` para o laço de eventos/agendamento interno; ``read`/`write`` para E/S da conexão HTTP.

7. O aumento da concorrência melhorou responsividade/vazão? A que custo?

Não de forma proporcional: no CPU, o throughput caiu de 0,503 req/s (nível 2) para ~0,09 req/s a partir do nível 4, indicando saturação; no GPU, houve uma queda acentuada de 1,667 req/s (nível 2) para ~0,37–0,47 req/s do nível 4 em diante, sinal de que uma única GPU serializa boa parte do trabalho sob carga simultânea. O custo, em ambos, foi memória crescente com o nível de concorrência.

8. Houve sinais de competição por recursos?

Sim. O container CPU manteve uso de CPU entre 1.370–1.410% (~14 núcleos) em todos os níveis de concorrência testados — saturação já a partir de 2 requisições simultâneas. O container GPU saltou de 146% (nível 2) para 620–750% de CPU a partir do nível 4, com a GPU em 60% de utilização — indicando competição tanto pela GPU quanto por CPU de suporte (roteamento MoE, pós-processamento) sob carga.

9. Como entradas maiores afetaram desempenho/recursos?

As perguntas mais longas (maior `eval_count`, ex. execuções 4 e 5) levaram consistentemente mais tempo, tanto em CPU quanto em GPU — a duração acompanhou de perto o número de tokens gerados dividido pela taxa de tokens/s de cada configuração, que se manteve relativamente estável (~140–195 tok/s na GPU, ~38–48 tok/s na CPU) independentemente do tamanho da entrada.

10. Como a arquitetura local contribui para privacidade/autonomia? Riscos remanescentes?

Toda a inferência ocorre localmente, sem envio de prompts a serviços de terceiros, dando controle total sobre os dados. Riscos remanescentes: ausência de criptografia em repouso nos volumes Docker, ausência de autenticação forte por padrão no Open WebUI, e dependência de um único ponto de execução sem redundância.

11. Como o sistema se comportaria com múltiplos usuários simultâneos?

Com base nos testes de concorrência, o container CPU já apresenta degradação severa a partir de 4 usuários simultâneos (throughput ~0,09 req/s, latência >20s). O container GPU, apesar de mais

rápido em termos absolutos, também apresenta queda de throughput já a partir de 4 requisições simultâneas (de 1,667 para ~0,41 req/s) — sugerindo que uma única GPU RTX 3060 tem paralelismo real limitado para atender muitos usuários ao mesmo tempo, mesmo com um modelo leve como o Granite MoE.

12. Quais limitações de hardware/runtime/aplicação afetam a validade?

Ver Seção 13 (Limitações e ameaças à validade).

13. Quais informações geradas por IA precisaram ser verificadas/corrigidas?

Ver Seção 14 (Declaração de Uso de IA Generativa).

11.1 Comparação com o experimento anterior (Phi-4-mini, modelo denso)

Métrica	Phi-4-mini (denso, 3,8B)	Granite MoE (3B totais / 800M ativos)
Speedup de duração GPU vs. CPU (sequencial)	5,21×	3,81×
Speedup de tokens/s GPU vs. CPU (sequencial)	7,54×	3,99×
Tokens/s médio em CPU	12,71	42,03
Tokens/s médio em GPU	95,88	167,74
VRAM ocupada durante inferência	≈4,1 GB (≈33%)	≈3,4 GB (≈27%)
Utilização de GPU durante inferência	87%	60%

Um resultado notável: apesar do Granite ser mais rápido em termos absolutos tanto em CPU quanto em GPU (maior tokens/s nas duas configurações), o ganho relativo de usar GPU foi menor com o Granite ($3,81 \times / 3,99 \times$) do que com o Phi-4-mini ($5,21 \times / 7,54 \times$). Isso é coerente com a arquitetura MoE do Granite: como apenas ~800M de parâmetros são ativados por token, a CPU já consegue processar essa fração de forma relativamente eficiente, reduzindo a vantagem proporcional da GPU em comparação com um modelo denso de tamanho semelhante, onde todos os parâmetros são sempre computados. Esse achado sustenta a hipótese, levantada antes da troca de modelo, de que "modelos menores ou com menos parâmetros ativos por token tendem a reduzir o ganho relativo da GPU, mesmo que ela continue mais rápida em termos absolutos".

12. Relação com Sistemas Operacionais e IA generativa

- Processos e threads: o Ollama demonstra um modelo clássico de servidor multi-thread (1 processo, 35–36 threads), ilustrando concorrência dentro de um único espaço de endereçamento — o mesmo padrão observado no experimento anterior com o Phi-4-mini.
- Escalonamento: a saturação de CPU observada no container CPU (≈1.400%, ~14 núcleos, já a partir de 2 requisições simultâneas) mostra o escalonador do Linux distribuindo threads entre núcleos até o limite físico disponível ao container.
- Sincronização: o domínio de `futex` no strace (74,16% do tempo) evidencia mecanismos de exclusão mútua/sincronização entre threads sob carga concorrente, reforçado pelo roteamento dinâmico entre "especialistas" característico da arquitetura MoE.
- Memória virtual: mesmo um modelo MoE precisa manter todos os "especialistas" mapeados em memória (RAM/VRAM), ainda que só uma fração seja computada por token — ilustrando a diferença entre memória alocada e trabalho computacional efetivo.
- Isolamento de recursos: os containers Docker demonstram uso de namespaces e cgroups do Linux para isolar CPU, memória, rede e (via NVIDIA Container Toolkit) acesso a dispositivo de GPU.
- Escalonamento de GPU: a queda de throughput já a partir de 4 requisições simultâneas (mesmo na configuração GPU) sugere que uma única GPU serializa parte do trabalho concorrente — um gargalo diferente do modelo de fila por núcleos observado na CPU.

13. Limitações e ameaças à validade

- Ambiente virtualizado (WSL2) introduz uma camada adicional entre o hardware físico e o Linux observado, podendo distorcer levemente medições de tempo em relação a um Linux nativo.
- Os recursos do WSL2 são compartilhados com o Windows host, que pode estar executando outros processos em segundo plano, afetando a estabilidade das medições.
- Cada nível de concorrência foi executado uma única vez, e não as três repetições recomendadas pela atividade — os resultados representam uma única amostra por configuração, sem desvio padrão.
- O conjunto de perguntas utilizado (5 perguntas, reaproveitadas nos testes de concorrência) é pequeno e pode não representar toda a variabilidade de cargas de trabalho reais.
- A rede host (--network=host) simplifica o setup mas reduz o isolamento de rede que existiria em um ambiente de produção com bridge networking.
- A captura de `strace` deste experimento ($\approx 0,31s$) foi muito mais curta que a do experimento anterior com o Phi-4-mini (200s), pois foi interrompida manualmente logo após a anexação — isso limita a comparação em número absoluto de chamadas entre os dois modelos, ainda que a proporção de `futex` permaneça comparável.

14. Declaração de Uso de IA Generativa

Ferramenta e modelo utilizados: Claude (Anthropic), utilizado como assistente de apoio ao longo da atividade.

Finalidade de cada uso:

- Apoio na definição da arquitetura experimental (dois containers Ollama isolados, um com e outro sem GPU, para comparação controlada).
- Geração dos scripts de benchmark (`benchmark_gpu_vs_cpu_granite.py`) e de geração de gráficos (`gerar_graficos_comparacao_granite.py`), adaptados do experimento anterior com o Phi-4-mini.
- Apoio na solução de problemas de instalação/execução (ex. conflito de porta 11434 já em uso, ausência do `strace` dentro da imagem do Ollama, conflito de nomes de container ao adotar `docker-compose`).
- Elaboração do `docker-compose.yml` e do script `coletar_inventario.sh`, e orientação sobre a troca de modelo reaproveitando a infraestrutura já criada.

Até cinco prompts relevantes utilizados (exemplos):

- "Na atividade pede para incluir no repositório `docker-compose`, `dockfiler` se tiver"
- "Gostaria de criar uma pasta inventario e um script para pegar os dados pedidos para esse inventário"
- "apareceu esse erro" (conflito de nome de container ao rodar `docker compose up -d`)
- "Faça um relatório com os dados... e se possível faça um README tbm para o repositório"

Testes, documentação ou observações usadas para verificar as respostas:

- O script de gráficos foi testado com dados fictícios antes de ser usado com dados reais, para confirmar que rodava sem erros.

- Os comandos de instalação (Docker, NVIDIA Container Toolkit) foram conferidos e efetivamente executados no ambiente WSL2 da equipe, com evidências capturadas em prints de tela.

15. Conclusão

Os experimentos confirmaram que a execução com GPU (NVIDIA RTX 3060) reduziu o tempo médio de resposta em cerca de $3,8\times$ e aumentou a geração de tokens por segundo em cerca de $4,0\times$ em relação à execução apenas em CPU, para o modelo Granite-3.1-3B-A800M-Instruct (MoE, 3B totais / $\sim 800M$ ativos, Q4_K_M). Sob carga concorrente, o container CPU satura já a partir de 4 requisições simultâneas (throughput caindo para $\sim 0,09$ req/s), e mesmo o container GPU apresenta queda expressiva de throughput no mesmo ponto (de 1,667 para $\sim 0,41$ req/s) — evidência de que uma única GPU serializa parte do trabalho concorrente. Comparado ao experimento anterior com o Phi-4-mini (modelo denso), o Granite foi mais rápido em termos absolutos (maior tokens/s em CPU e em GPU), mas apresentou um ganho relativo de GPU menor ($3,8\times/4,0\times$ contra $5,2\times/7,5\times$ do Phi-4-mini) — um resultado consistente com a natureza da arquitetura MoE, que ativa apenas uma fração dos parâmetros por token e por isso já é relativamente eficiente mesmo em CPU.

[PREENCHER: reflexões adicionais da equipe: principais aprendizados, dificuldades encontradas e como a atividade contribuiu para entender conceitos de SO na prática]

16. Referências

- OLLAMA. Documentação oficial. Disponível em: <https://docs.ollama.com>
- OPEN WEBUI. Documentação oficial. Disponível em: <https://docs.openwebui.com>
- IBM. Granite-3.1-3B-A800M-Instruct — Model Card. Hugging Face. Disponível em: <https://huggingface.co/ibm-granite/granite-3.1-3b-a800m-instruct>
- MICROSOFT. Phi-4-mini-instruct — Model Card (modelo do experimento anterior, citado na Seção 11.1). Hugging Face. Disponível em: <https://huggingface.co/microsoft/Phi-4-mini-instruct>
- NVIDIA. NVIDIA Container Toolkit — Installation Guide. Disponível em: <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/>
- MICROSOFT. Documentação do WSL2 (Windows Subsystem for Linux). Disponível em: <https://learn.microsoft.com/windows/wsl/>
- DOCKER. Compose file reference. Disponível em: <https://docs.docker.com/compose/compose-file/>

[PREENCHER: demais referências utilizadas pela equipe (artigos, documentação adicional etc.)]